CSS Complete Reference

All Properties · Selectors · Pseudo-classes/elements · Flexbox · Grid · Animations · At-rules

Covering modern CSS — Selectors, Box Model, Typography, Colors, Flexbox, Grid, Animations, Variables, Responsive Design, Container Queries, and more.

Table of Contents

- 1. Selectors (14 items)
- 2. Pseudo-classes (24 items)
- 3. Pseudo-elements (9 items)
- 4. Box Model (19 items)
- 5. Typography (23 items)
- 6. Colors & Backgrounds (18 items)
- 7. Layout — Display & Positioning (14 items)
- 8. Flexbox (17 items)
- 9. CSS Grid (21 items)
- 10. Transitions & Animations (16 items)
- 11. CSS Variables & Functions (16 items)
- 12. Responsive Design (14 items)
- 13. Spacing, Sizing & Overflow (15 items)
- 14. At-rules & Other (16 items)

1. Selectors

Property / Selector	Explanation	Example
*	Universal selector — selects every element	`* { margin: 0; }`
element	Type selector — selects all matching HTML elements	`p { color: red; }`
.class	Class selector — selects elements with the given class	`.card { padding: 10px; }`
#id	ID selector — selects the element with the given id	`#header { background: navy; }`
A, B	Group selector — applies rules to multiple selectors	`h1, h2 { font-weight: bold; }`
A B	Descendant combinator — B inside A (any depth)	`div p { color: blue; }`
A > B	Child combinator — direct children only	`ul > li { list-style: none; }`
A + B	Adjacent sibling — B immediately after A	`h1 + p { margin-top: 0; }`
A ~ B	General sibling — all B after A	`h1 ~ p { color: gray; }`
[attr]	Attribute selector — element has the attribute	`input[required] { border: red; }`
[attr=val]	Attribute equals value	`input[type='text'] { border: 1px solid; }`
[attr^=val]	Attribute value starts with val	`a[href^='https'] { color: green; }`
[attr$=val]	Attribute value ends with val	`a[href$='.pdf'] { color: red; }`
[attr*=val]	Attribute value contains val	`a[href*='google'] { color: blue; }`

2. Pseudo-classes

Property / Selector	Explanation	Example
:hover	Styles an element when the mouse is over it	`a:hover { color: orange; }`
:focus	Styles a focused element (keyboard/click)	`input:focus { outline: 2px solid blue; }`
:active	Styles an element being clicked	`button:active { background: gray; }`
:visited	Styles already-visited links	`a:visited { color: purple; }`
:link	Styles unvisited links	`a:link { color: blue; }`
:nth-child(n)	Selects element that is the nth child	`tr:nth-child(2n) { background: #eee; }`
:nth-of-type(n)	Selects the nth sibling of its type	`p:nth-of-type(1) { font-weight: bold; }`
:first-child	Selects element that is the first child	`li:first-child { color: red; }`
:last-child	Selects the last child element	`li:last-child { border: none; }`
:only-child	Selects an element with no siblings	`p:only-child { text-align: center; }`
:not(selector)	Selects elements NOT matching the selector	`p:not(.intro) { color: gray; }`
:checked	Targets checked checkboxes or radio buttons	`input:checked { accent-color: green; }`

2. Pseudo-classes

Property / Selector	Explanation	Example
:disabled	Targets disabled form elements	`input:disabled { opacity: 0.5; }`
:enabled	Targets enabled form elements	`input:enabled { background: white; }`
:empty	Selects elements with no children	`div:empty { display: none; }`
:root	Selects the root element (); used for CSS vars	`:root { --primary: #333; }`
:is(A, B)	Matches any selector in the list (forgiving)	`:is(h1,h2,h3) { font-family: sans-serif; }`
:where(A, B)	Like :is() but with zero specificity	`:where(article, section) > p { line-height: 1.6; }`
:has(A)	Parent selector — matches element that has child A	`div:has(> img) { border: 1px solid; }`
:target	Matches the element targeted by the URL fragment	`#section:target { background: yellow; }`
:lang(code)	Matches elements in a specific language	`p:lang(fr) { quotes: '\AB' '\BB'; }`
:placeholder-shown	Matches input when placeholder is visible	`input:placeholder-shown { border-color: #ccc; }`
:focus-within	Matches element if any descendant has focus	`form:focus-within { box-shadow: 0 0 4px blue; }`
:focus-visible	Focus only when navigating by keyboard	`button:focus-visible { outline: 3px solid blue; }`

3. Pseudo-elements

Property / Selector	Explanation	Example
::before	Inserts content before an element's content	`p::before { content: '★ '; }`
::after	Inserts content after an element's content	`p::after { content: ' ←'; color: red; }`
::first-line	Styles the first line of a block element	`p::first-line { font-weight: bold; }`
::first-letter	Styles the first letter of a block element	`p::first-letter { font-size: 2em; float: left; }`
::placeholder	Styles the placeholder text of inputs	`input::placeholder { color: #aaa; font-style: italic; }`
::selection	Styles text that the user has selected	`::selection { background: yellow; color: black; }`
::marker	Styles list-item markers (bullet/number)	`li::marker { color: red; font-size: 1.2em; }`
::backdrop	Styles the backdrop behind a	`dialog::backdrop { background: rgba(0,0,0,0.5); }`
::file-selector-button	Styles the button inside a file input	`input[type=file]::file-selector-button { background: blue; }`

4. Box Model

Property / Selector	Explanation	Example
width	Sets the width of an element's content area	`div { width: 300px; }`
height	Sets the height of an element's content area	`div { height: 200px; }`
min-width	Minimum width an element can shrink to	`img { min-width: 100px; }`
max-width	Maximum width an element can grow to	`.container { max-width: 1200px; }`
min-height	Minimum height an element can shrink to	`section { min-height: 100vh; }`
max-height	Maximum height; overflow hidden beyond this	`.card { max-height: 400px; overflow: auto; }`
padding	Space inside the element between border and content	`div { padding: 10px 20px; }`
padding-top/right/bottom/left	Individual padding sides	`p { padding-top: 8px; padding-left: 16px; }`
margin	Space outside the element (outside border)	`p { margin: 0 auto; }`
margin-top/right/bottom/left	Individual margin sides	`h1 { margin-bottom: 24px; }`
border	Shorthand for border-width, border-style, border-color	`div { border: 2px solid black; }`
border-width	Thickness of the border	`input { border-width: 1px; }`
border-style	Style: solid, dashed, dotted, double, none, etc.	`div { border-style: dashed; }`
border-color	Color of the border	`div { border-color: red; }`
border-radius	Rounds the corners of an element	`button { border-radius: 8px; }`
border-top/right/bottom/left	Individual border sides	`p { border-bottom: 1px solid #ccc; }`
box-sizing	content-box or border-box (includes padding/border)	`* { box-sizing: border-box; }`
outline	Outline drawn outside the border (no layout shift)	`a:focus { outline: 2px solid blue; }`
outline-offset	Distance between outline and border edge	`a:focus { outline-offset: 4px; }`

5. Typography

Property / Selector	Explanation	Example
font-family	Sets the typeface; provide fallbacks	`body { font-family: 'Roboto', sans-serif; }`
font-size	Sets the text size	`p { font-size: 16px; }`
font-weight	Boldness: normal, bold, 100–900	`h1 { font-weight: 700; }`
font-style	normal, italic, or oblique	`em { font-style: italic; }`
font-variant	small-caps and other variants	`abbr { font-variant: small-caps; }`

5. Typography

Property / Selector	Explanation	Example
font	Shorthand for font-style/variant/weight/size/family	`p { font: bold 16px/1.5 Arial, sans-serif; }`
line-height	Height of each line of text	`p { line-height: 1.6; }`
letter-spacing	Space between characters	`h2 { letter-spacing: 0.1em; }`
word-spacing	Space between words	`p { word-spacing: 4px; }`
text-align	Horizontal alignment: left, right, center, justify	`p { text-align: justify; }`
text-decoration	Underline, overline, line-through, none	`a { text-decoration: none; }`
text-transform	uppercase, lowercase, capitalize, none	`button { text-transform: uppercase; }`
text-indent	Indents the first line of a paragraph	`p { text-indent: 2em; }`
text-shadow	Shadow behind text (x y blur color)	`h1 { text-shadow: 2px 2px 4px rgba(0,0,0,0.3); }`
text-overflow	Handles text that overflows its container	`p { text-overflow: ellipsis; overflow: hidden; white-space: nowrap; }`
white-space	Controls how whitespace is handled	`pre { white-space: pre-wrap; }`
word-break	Controls line breaks within words	`p { word-break: break-word; }`
overflow-wrap	Breaks long words to prevent overflow	`div { overflow-wrap: anywhere; }`
hyphens	Enables automatic hyphenation	`p { hyphens: auto; }`
vertical-align	Aligns inline or table-cell elements vertically	`img { vertical-align: middle; }`
direction	ltr or rtl text direction	`p { direction: rtl; }`
unicode-bidi	Overrides bidirectional text algorithm	`p { unicode-bidi: embed; }`
-webkit-font-smoothing	Antialiasing on macOS/iOS	`body { -webkit-font-smoothing: antialiased; }`

6. Colors & Backgrounds

Property / Selector	Explanation	Example
color	Sets the foreground (text) color	`p { color: #333; }`
background-color	Sets the background fill color	`div { background-color: #f0f4ff; }`
background-image	Sets one or more background images/gradients	`div { background-image: url('hero.jpg'); }`
background-repeat	How the background image tiles	`div { background-repeat: no-repeat; }`
background-position	Position of the background image	`div { background-position: center top; }`
background-size	cover, contain, or explicit size	`div { background-size: cover; }`
background-attachment	fixed, scroll, or local	`body { background-attachment: fixed; }`
background-origin	padding-box, border-box, or content-box	`div { background-origin: content-box; }`
background-clip	Clips the background to an area	`h1 { background-clip: text; }`

6. Colors & Backgrounds

Property / Selector	Explanation	Example
background	Shorthand for all background properties	`div { background: #fff url('bg.png') no-repeat center/cover; }`
opacity	Sets element transparency (0–1)	`img { opacity: 0.8; }`
linear-gradient()	Creates a linear color gradient	`div { background: linear-gradient(45deg, #ff6b6b, #4ecdc4); }`
radial-gradient()	Creates a radial color gradient	`div { background: radial-gradient(circle, white, black); }`
conic-gradient()	Creates a conic (pie) gradient	`div { background: conic-gradient(red 90deg, blue 90deg); }`
filter	Applies visual filters: blur, brightness, contrast…	`img { filter: grayscale(100%); }`
backdrop-filter	Filter applied to content behind element	`.modal { backdrop-filter: blur(8px); }`
mix-blend-mode	How an element blends with its background	`img { mix-blend-mode: multiply; }`
mask-image	Masks the element with an image or gradient	`div { mask-image: linear-gradient(black, transparent); }`

7. Layout — Display & Positioning

Property / Selector	Explanation	Example
display	Determines layout mode: block/inline/flex/grid/none…	`div { display: flex; }`
position	static, relative, absolute, fixed, sticky	`div { position: absolute; top: 0; left: 0; }`
top / right / bottom / left	Offsets for positioned elements	`.tooltip { position: absolute; top: 100%; left: 0; }`
z-index	Stacking order (higher = on top) for positioned elements	`.modal { position: fixed; z-index: 1000; }`
float	Floats element left or right; text wraps around it	`img { float: left; margin-right: 16px; }`
clear	Prevents wrapping next to floated elements	`.footer { clear: both; }`
overflow	Handles content that overflows: visible/hidden/scroll/auto	`div { overflow: hidden; }`
overflow-x / overflow-y	Overflow on horizontal or vertical axis	`pre { overflow-x: auto; }`
visibility	visible or hidden (space still reserved)	`.hidden { visibility: hidden; }`
clip-path	Clips element to a geometric shape	`img { clip-path: circle(50%); }`
transform	2D/3D transforms: translate, rotate, scale, skew	`div { transform: rotate(45deg) scale(1.2); }`
transform-origin	Sets the pivot point for transforms	`img { transform-origin: top left; }`
perspective	Sets perspective depth for 3D transforms	`.scene { perspective: 800px; }`

7. Layout — Display & Positioning

Property / Selector	Explanation	Example
backface-visibility	Shows/hides back face during 3D rotation	`.card { backface-visibility: hidden; }`

8. Flexbox

Property / Selector	Explanation	Example
display: flex	Turns a container into a flex container	`.container { display: flex; }`
flex-direction	row (default), row-reverse, column, column-reverse	`.nav { flex-direction: row; }`
flex-wrap	nowrap/wrap/wrap-reverse — allows multi-line flex	`.grid { flex-wrap: wrap; }`
flex-flow	Shorthand for flex-direction + flex-wrap	`.box { flex-flow: row wrap; }`
justify-content	Aligns items on the main axis	`.bar { justify-content: space-between; }`
align-items	Aligns items on the cross axis	`.bar { align-items: center; }`
align-content	Aligns multiple rows on the cross axis	`.wrap { align-content: flex-start; }`
gap	Shorthand gap between flex/grid items	`.grid { gap: 16px; }`
row-gap / column-gap	Individual row or column gaps	`.grid { row-gap: 8px; column-gap: 16px; }`
order	Changes the visual order of a flex item (default 0)	`.sidebar { order: -1; }`
flex-grow	How much a flex item grows relative to others	`.main { flex-grow: 1; }`
flex-shrink	How much a flex item shrinks relative to others	`.logo { flex-shrink: 0; }`
flex-basis	Initial main size before growing/shrinking	`.col { flex-basis: 200px; }`
flex	Shorthand for flex-grow, flex-shrink, flex-basis	`.item { flex: 1 1 auto; }`
align-self	Overrides align-items for a single flex item	`.special { align-self: flex-end; }`
place-items	Shorthand for align-items + justify-items	`.card { place-items: center; }`
place-content	Shorthand for align-content + justify-content	`.wrapper { place-content: center; }`

9. CSS Grid

Property / Selector	Explanation	Example
display: grid	Turns a container into a grid container	`.layout { display: grid; }`
grid-template-columns	Defines column tracks	`.grid { grid-template-columns: repeat(3, 1fr); }`
grid-template-rows	Defines row tracks	`.grid { grid-template-rows: auto 1fr auto; }`
grid-template-areas	Defines named grid areas	`.layout { grid-template-areas: 'header header' 'sidebar main'; }`

9. CSS Grid

Property / Selector	Explanation	Example
grid-template	Shorthand for columns, rows, and areas	`.g { grid-template: auto 1fr / 200px 1fr; }`
grid-column-gap (column-gap)	Space between columns	`.grid { column-gap: 24px; }`
grid-row-gap (row-gap)	Space between rows	`.grid { row-gap: 16px; }`
grid-gap (gap)	Shorthand row and column gap	`.grid { gap: 16px 24px; }`
grid-column	Shorthand for column start/end line placement	`.hero { grid-column: 1 / -1; }`
grid-row	Shorthand for row start/end line placement	`.sidebar { grid-row: 2 / 4; }`
grid-area	Assigns an item to a named grid area	`.header { grid-area: header; }`
grid-auto-columns	Size of implicitly created column tracks	`.grid { grid-auto-columns: minmax(100px, 1fr); }`
grid-auto-rows	Size of implicitly created row tracks	`.grid { grid-auto-rows: minmax(80px, auto); }`
grid-auto-flow	Controls auto-placement: row/column/dense	`.mosaic { grid-auto-flow: dense; }`
justify-items	Aligns items inside cells on inline axis	`.grid { justify-items: center; }`
align-items	Aligns items inside cells on block axis	`.grid { align-items: start; }`
justify-self	Overrides justify-items for a single item	`.item { justify-self: end; }`
align-self	Overrides align-items for a single grid item	`.item { align-self: stretch; }`
fr unit	Fractional unit — divides remaining space	`.g { grid-template-columns: 2fr 1fr; }`
minmax()	Sets min and max size for a grid track	`.g { grid-template-columns: minmax(200px, 1fr) 1fr; }`
repeat()	Repeats track definitions	`.g { grid-template-columns: repeat(auto-fill, minmax(150px,1fr)); }`

10. Transitions & Animations

Property / Selector	Explanation	Example
transition	Shorthand: property duration timing-function delay	`button { transition: background 0.3s ease; }`
transition-property	Which CSS property to animate	`a { transition-property: color, opacity; }`
transition-duration	How long the transition takes	`div { transition-duration: 500ms; }`
transition-timing-function	Speed curve: ease, linear, ease-in, ease-out, cubic-bezier	`div { transition-timing-function: ease-in-out; }`
transition-delay	Wait before transition starts	`div { transition-delay: 200ms; }`
animation	Shorthand for all animation sub-properties	`div { animation: spin 1s linear infinite; }`
animation-name	Name of the @keyframes rule to use	`div { animation-name: fadeIn; }`

10. Transitions & Animations

Property / Selector	Explanation	Example
animation-duration	How long one cycle of animation takes	`div { animation-duration: 2s; }`
animation-timing-function	Speed curve of the animation	`div { animation-timing-function: ease-in; }`
animation-delay	Delay before animation starts	`div { animation-delay: 0.5s; }`
animation-iteration-count	Times to repeat: number or infinite	`div { animation-iteration-count: infinite; }`
animation-direction	normal, reverse, alternate, alternate-reverse	`div { animation-direction: alternate; }`
animation-fill-mode	Styles before/after animation: forwards/backwards/both	`div { animation-fill-mode: forwards; }`
animation-play-state	running or paused	`div:hover { animation-play-state: paused; }`
@keyframes	Defines the animation's key states	`@keyframes spin { from { rotate: 0deg; } to { rotate: 360deg; } }`
will-change	Hints the browser to prepare for changes	`.box { will-change: transform; }`

11. CSS Variables & Functions

Property / Selector	Explanation	Example
--variable-name	Declares a custom property (CSS variable)	`:root { --clr-primary: #3949ab; }`
var()	Accesses a CSS variable (with optional fallback)	`button { background: var(--clr-primary, blue); }`
calc()	Performs arithmetic with mixed units	`div { width: calc(100% - 2rem); }`
min()	Returns the smallest of two values	`p { font-size: min(4vw, 20px); }`
max()	Returns the largest of two values	`.hero { height: max(60vh, 400px); }`
clamp()	Clamps a value between min and max	`h1 { font-size: clamp(1.5rem, 5vw, 3rem); }`
env()	Accesses environment variables (safe-area-inset…)	`body { padding-bottom: env(safe-area-inset-bottom); }`
attr()	Reads an HTML attribute value into CSS	`a::after { content: attr(href); }`
counter()	Inserts CSS counter value as content	`li::before { content: counter(item); }`
counters()	Nested counters for sub-lists	`li::before { content: counters(i,'.'); }`
url()	References an external resource	`div { background: url('pattern.svg'); }`
format()	Specifies font format inside @font-face	`@font-face { src: url(f.woff2) format('woff2'); }`
rgb() / rgba()	Defines colors with optional alpha channel	`div { background: rgba(0,0,255,0.4); }`
hsl() / hsla()	Defines colors using hue/saturation/lightness	`p { color: hsl(220, 70%, 50%); }`
hwb()	Hue / whiteness / blackness color model	`span { color: hwb(220 20% 10%); }`
oklch() / oklab()	Perceptually uniform color spaces (modern)	`a { color: oklch(60% 0.2 250); }`

12. Responsive Design

Property / Selector	Explanation	Example
@media (max-width)	Applies styles for screens up to a width	`@media (max-width: 768px) { .nav { display: none; } }`
@media (min-width)	Applies styles for screens at least as wide	`@media (min-width: 1024px) { .col { width: 33%; } }`
@media print	Applies styles only when printing	`@media print { .ads { display: none; } }`
@media (orientation)	Targets portrait or landscape orientation	`@media (orientation: landscape) { .hero { height: 60vh; } }`
@media (prefers-color-scheme)	Detects dark or light mode preference	`@media (prefers-color-scheme: dark) { body { background: #111; } }`
@media (prefers-reduced-motion)	Reduces animations for users who prefer it	`@media (prefers-reduced-motion) { * { animation: none !important; } }`
@media (hover)	Checks if device supports hover interactions	`@media (hover: hover) { a:hover { color: blue; } }`
@container	Container query — styles based on parent size	`@container (min-width: 400px) { .card { flex-direction: row; } }`
container-type	Defines a containment context for container queries	`.wrapper { container-type: inline-size; }`
@layer	Defines cascade layers for specificity management	`@layer base, components, utilities;`
@supports	Applies styles only if a feature is supported	`@supports (display: grid) { .layout { display: grid; } }`
viewport units (vw/vh/vmin/vmax)	Relative to viewport dimensions	`section { height: 100vh; padding: 5vw; }`
dvh / svh / lvh	Dynamic/small/large viewport height (mobile-safe)	`.hero { height: 100dvh; }`
cqi / cqw	Container-query inline/block size units	`h2 { font-size: 5cqi; }`

13. Spacing, Sizing & Overflow

Property / Selector	Explanation	Example
aspect-ratio	Sets a preferred aspect ratio for the element	`img { aspect-ratio: 16 / 9; }`
object-fit	How img/video fits its box: fill/contain/cover/none	`img { object-fit: cover; width: 100%; height: 200px; }`
object-position	Position of replaced content inside its box	`img { object-position: top center; }`
resize	Allows the user to resize the element	`textarea { resize: vertical; }`
scroll-behavior	Smooth or instant scrolling when navigating	`html { scroll-behavior: smooth; }`
scroll-margin	Offset for scroll-snapping / anchor targets	`section { scroll-margin-top: 80px; }`
scroll-padding	Adjusts scroll container's snap target area	`html { scroll-padding-top: 80px; }`

13. Spacing, Sizing & Overflow

Property / Selector	Explanation	Example
scroll-snap-type	Activates scroll snapping on a container	`.slider { scroll-snap-type: x mandatory; }`
scroll-snap-align	How children snap: start, center, end	`.slide { scroll-snap-align: start; }`
overscroll-behavior	Prevents scroll chaining to parent	`.modal { overscroll-behavior: contain; }`
touch-action	Controls which touch gestures the browser handles	`.map { touch-action: pan-x pan-y; }`
user-select	Controls text-selection behavior	`button { user-select: none; }`
pointer-events	none disables all mouse/touch events on element	`.overlay { pointer-events: none; }`
cursor	Sets the mouse cursor icon	`button { cursor: pointer; }`
caret-color	Color of the text-input caret	`input { caret-color: blue; }`

14. At-rules & Other

Property / Selector	Explanation	Example
@import	Imports another CSS file	`@import url('reset.css');`
@font-face	Defines a custom font to use on the page	`@font-face { font-family: 'MyFont'; src: url('f.woff2') format('woff2'); }`
@keyframes	Defines animation states for the animation property	`@keyframes fadeIn { from { opacity:0; } to { opacity:1; } }`
@media	Applies styles under specific media conditions	`@media screen and (max-width: 600px) { body { font-size: 14px; } }`
@supports	Applies styles only if browser supports a feature	`@supports (gap: 1rem) { .flex { gap: 1rem; } }`
@layer	Organises styles into named cascade layers	`@layer utilities { .mt-4 { margin-top: 1rem; } }`
@container	Container query block (needs container-type on parent)	`@container sidebar (min-width: 300px) { .widget { flex: row; } }`
@property	Registers a custom property with type/syntax info	`@property --hue { syntax: ''; inherits: false; initial-value: 0deg; }`
@counter-style	Defines custom list marker styles	`@counter-style thumbs { system: cyclic; symbols: '\1F44D'; suffix: ' '; }`
@namespace	Declares an XML namespace (SVG/MathML in HTML)	`@namespace svg url('http://www.w3.org/2000/svg');`
!important	Overrides any other declarations for a property	`.hidden { display: none !important; }`
inherit	Forces a property to inherit from its parent	`button { color: inherit; }`
initial	Resets a property to its CSS initial value	`h1 { font-size: initial; }`
unset	Unsets to inherited value (or initial if none)	`p { border: unset; }`

14. At-rules & Other

Property / Selector	Explanation	Example
revert	Reverts to browser default stylesheet value	`a { color: revert; }`
revert-layer	Reverts to value from the previous cascade layer	`a { color: revert-layer; }`